

Comunicación simultánea y disponibilidad del servicio

4. Comunicación simultánea y disponibilidad del servicio

Objetivo de la sesión

Cerramos el tema mirando el servicio desde dos perspectivas distintas.

1 Concurrency

¿Puede atender varias peticiones a la vez sin procesarlas necesariamente de forma secuencial?

2 Disponibilidad

¿Cómo sabemos automáticamente si la aplicación y sus dependencias siguen realmente sanas?

Idea clave: Una petición correcta no basta para describir el comportamiento operativo del servicio.

Concurrencia: varias peticiones a la vez

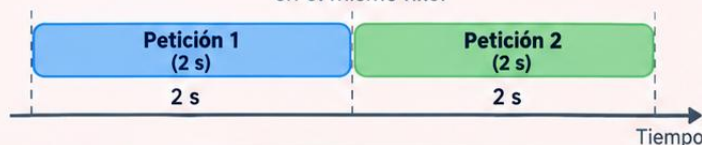
Cómo atiende Tomcat varias peticiones a la vez

Pool de hilos y procesamiento concurrente en un servidor Java



Atención secuencial

Las peticiones se atienden una detrás de otra en el mismo hilo.

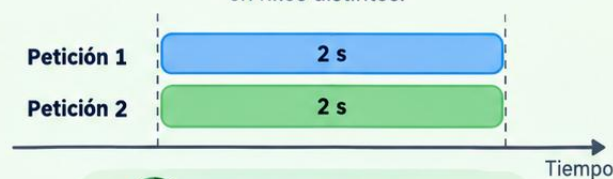


Tiempo total ≈ 4 s

Una espera a que termine la otra.

Atención en paralelo

Las peticiones se atienden al mismo tiempo en hilos distintos.



Tiempo total ≈ 2 s

Se ejecutan al mismo tiempo en hilos distintos.



Idea clave: Tomcat no crea un proceso por petición; utiliza un **pool de hilos** para gestionar varias solicitudes concurrentes.

Cómo lo vas a comprobar

Simularemos una operación lenta de 2 segundos y lanzaremos dos peticiones simultáneas.

1 Simular lentitud

```
Thread.sleep(2000);
```

2 Lanzar a la vez

```
time (curl -s http://localhost:8080/api/v1/libros/top & \  
      curl -s http://localhost:8080/api/v1/libros/top & \  
      wait)
```

Secuencial
≈ 4 s

Concurrente
≈ 2 s

Compruébalo en el log con `Thread.currentThread().getName()`: deberían aparecer hilos como `http-nio-8080-exec-1` y `exec-2`.

Disponibilidad: no basta con que el proceso exista

¿Qué significa que un servicio esté **disponible**?

No basta con que el proceso exista: la disponibilidad tiene varios niveles.



Health check

Una máquina comprueba **automáticamente** el estado del servicio cada pocos segundos.



Monitor

Comprueba
cada pocos segundos



Servicio

- ✓ ¿Responde?
- ✓ ¿Dependencias OK?
- ✓ ¿Listo para servir peticiones?



Idea clave: responder una petición no siempre significa que el servicio esté sano.

Spring Boot Actuator: añadir la comprobación de salud

Dependencia Maven

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Con solo esta dependencia, Spring Boot expone automáticamente:

`/actuator/health`

Detalles en desarrollo

```
management:
  endpoint:
    health:
      show-details: always
```

⚠ `show-details: always` expone información interna.

Úsalo en local; en producción suele ocultarse o restringirse.

Cómo /actuator/health agrega el estado

Cómo Spring Boot Actuator comprueba la salud de la aplicación

El endpoint /actuator/health agrega el estado de la aplicación y de sus dependencias



Idea clave: responder peticiones no basta; Actuator ayuda a saber si la aplicación y sus dependencias están realmente sanas.

Leer la respuesta de /actuator/health

```
{  
  "status": "UP",  
  "components": {  
    "db": { "status": "UP" },  
    "mongo": { "status": "UP" }  
  }  
}
```

UP

Aplicación y componentes comprobados disponibles.

DOWN

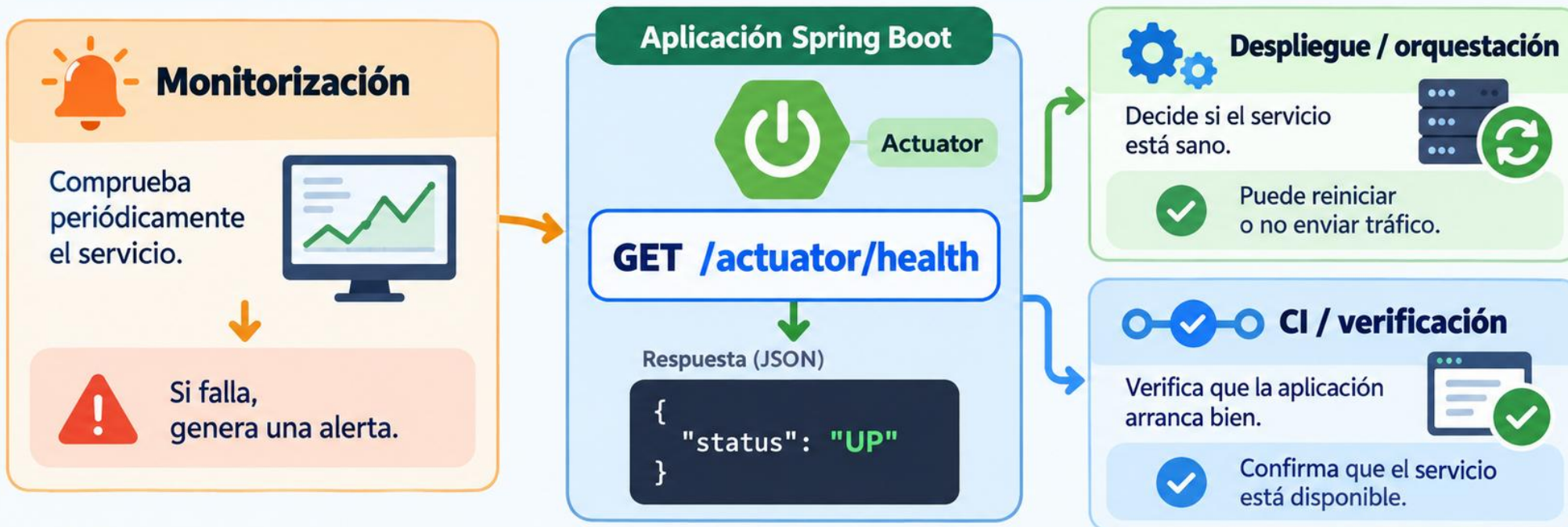
Una dependencia puede hacer caer el estado agregado.

**Responder un endpoint concreto
≠
estar completamente sano.**

Quién consulta /actuator/health y para qué

Quién consulta **/actuator/health** y para qué sirve

No es un endpoint pensado para usuarios finales, sino para comprobar automáticamente la salud del servicio.



La diferencia no es el protocolo: sigue siendo HTTP. Lo importante es **quién lo consulta y para qué**.



Idea clave

Una petición concreta puede funcionar, pero **/actuator/health** ayuda a saber **si la aplicación y sus dependencias están realmente sanas**.

Actuator: información operativa, no recursos de negocio

Otros endpoints útiles de Spring Boot Actuator

Información operativa para observar la aplicación, no recursos de negocio.



La API REST sirve recursos al cliente; Actuator ayuda a observar el estado interno y operativo del servicio.

Actividad 1.4: comprobar concurrencia y disponibilidad

Dos pequeñas pruebas sobre el mismo servicio ya construido.

1 Forzar una operación lenta

Añadir temporalmente `Thread.sleep(2000)` en el método indicado.

2 Lanzar dos peticiones simultáneas

Medir el tiempo conjunto y contrastar ≈ 4 s frente a ≈ 2 s.

3 Añadir Actuator

Incorporar la dependencia y activar los detalles de health en desarrollo.

4 Provocar una dependencia DOWN

Observar cómo cambia `/actuator/health` y localizar el componente responsable.

Evidencias útiles: tiempo medido + log de hilos + respuesta de `/actuator/health` antes/después del fallo.

Lo mínimo que debe quedar claro

- Tomcat utiliza un pool de hilos: varias peticiones pueden avanzar concurrentemente.
- Proceso vivo, responder peticiones y tener dependencias sanas son niveles distintos.
- Un health check está pensado para ser consultado automáticamente por otras máquinas.
- Actuator expone `/actuator/health` y puede agregar el estado de dependencias reales.
- `show-details`: always ayuda en desarrollo, pero revela información interna.
- `/actuator/info` y `/actuator/metrics` amplían la observabilidad operativa del servicio.

Cierre del Tema 1: de construir la API a comprobar que responde, resiste y puede observarse.